Chapter 12 — Testing Spring Applications

A Spring application is plain Java objects plus a **container** (Spring's *application context*: the object that creates your **beans**, i.e. the objects Spring manages, and plugs them into each other) that wires them together, adds **proxies** (wrapper objects that add behaviour such as transactions around your real object) and talks to the outside world (HTTP, databases, other services). Good tests match that shape. Most of your logic can be tested **without Spring at all**. Some things only exist inside the container: URL mappings, JSON conversion, validation, SQL generated by Spring Data, **transactions** (a group of database changes that succeed or fail together), security filters. For those you start **just enough of the container** to run them. A few tests start **everything** and send real HTTP requests to prove the parts fit together.

This chapter shows how to do each of these with Spring Boot 4's test support, and how to keep the **suite** (the whole collection of tests in a project) fast as it grows. Every behaviour is proved by a test whose real output is shown.

What you'll learn

1. The test pyramid for a Spring app, with real timings for each layer.
2. The Boot 4 test starters (one per technology) and what `spring-boot-starter-test` contains (JUnit 6, AssertJ, Mockito, ...).
3. How the **Spring TestContext Framework** loads, injects and caches application contexts.
4. `@SpringBootTest` and its four web environments.
5. **Test slices**: `@WebMvcTest`, `@DataJpaTest`, `@JsonTest`, `@RestClientTest` and the rest.
6. `MockMvcTester` (AssertJ-style MockMvc) and `RestTestClient` (new in Framework 7.0).
7. `@MockitoBean` / `@MockitoSpyBean`, which replaced Boot's `@MockBean` (removed in Boot 4).
8. **Context caching**: what makes a new context, measured hit/miss counts and timings, `@DirtiesContext`, context pausing (Framework 7.0).
9. `@DynamicPropertySource` for values only known at runtime (random ports, container URLs).
10. **Testcontainers +** `@ServiceConnection` for real databases in tests (shown, not run: needs Docker).
11. Test profiles and test properties.
12. **Types of API testing** (§12.18): smoke, functional, integration, contract, regression, load/stress/soak, security, fuzz and UI — the question each answers and the Spring or Java tool for it, with a runnable fuzz test.

All code is in `code/`. The projects use Spring Boot **4.1.1** (Framework 7.0, JUnit 6.0 — the framework that finds and runs Java tests, Mockito 5 — a library that creates fake objects, AssertJ 3 — a library of readable `assertThat(...)` checks) and Java 25:

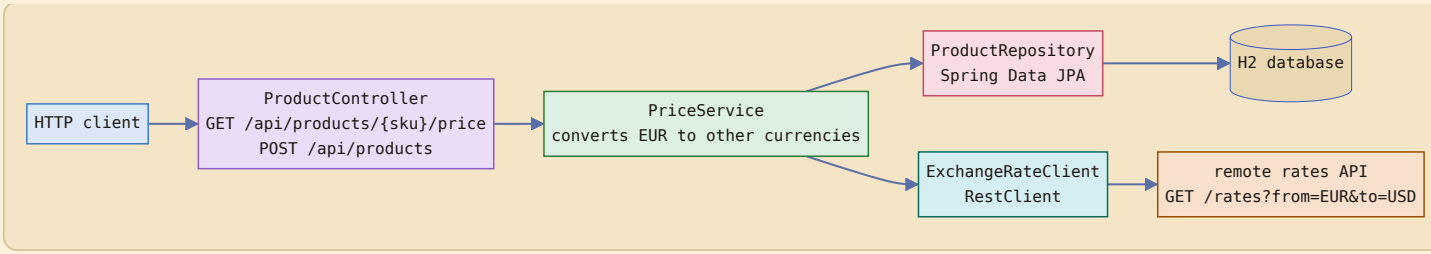
Project	Shows	Run here?
`testing-demo`	a small product-pricing API tested at every level: unit, slices, full context, real port, plus a seeded fuzz test	yes
`context-caching`	seven test classes that hit or miss the context cache, with timings	yes
`testcontainers-demo`	PostgreSQL (a popular open-source SQL database) in a container via `@ServiceConnection`	**needs Docker — not run** (compiled only)

Output blocks are real `mvn -q -B -ntp verify` output (Maven, the Java build tool, compiling the code and running every test). Spring's log lines are trimmed to `...`. **Timings are machine-dependent** (these came from a 14-core Apple M4 Pro laptop); only compare them with each other.

> **Prerequisites.** JUnit 5/6, Mockito and AssertJ basics are taught in the Java handbook, Chapter 18. This chapter assumes them and focuses on what Spring adds. The code under test uses Spring MVC (Chapter 5), `RestClient` (Chapter 6) and Spring Data JPA (Chapter 8). Security testing (`@WithMockUser`, `jwt()`) is in Chapter 10 and Chapter 11. Reactive testing (`StepVerifier`, `WebTestClient`) is in Chapter 13.

12.1 The application under test

`testing-demo` is a tiny catalogue service with the layers a real service has:



- `ProductController` maps HTTP to Java, validates the request body (`@Valid`) and turns `ProductNotFoundException` into a `404` Problem Detail (a standard JSON shape for API errors).
- `PriceService` holds the business rule: prices are stored in EUR; for another currency, fetch the rate and round to 2 decimals (`HALF_EVEN`, "banker's rounding": a 5 rounds towards the even neighbour). For EUR it must **not** call the remote API.
- `ProductRepository` is a Spring Data JPA interface (JPA = Java Persistence API, the standard for mapping Java objects to database tables) with a **derived query** (`findBySku`: Spring writes the SQL from the method name) and a **JPQL** query (`cheaperThan`: JPA's SQL-like language that talks about entity classes instead of tables).
- `ExchangeRateClient` calls a remote API with `RestClient`. Its base URL comes from the property `rates.base-url`.

The service, for reference:

```
@Service
public class PriceService {
    private final ProductRepository products;
    private final ExchangeRateClient rates;
    private final String home;

    public PriceService(ProductRepository products, ExchangeRateClient rates,
                        @Value("${catalog.home-currency}") String home) { ... }

    public PriceView priceOf(String sku, String currency) {
        Product p = products.findBySku(sku).orElseThrow(() -> new ProductNotFoundException(sku));
        BigDecimal amount = currency.equals(home)
                ? p.getPrice()                                  // no remote call needed
                : convert(p.getPrice(), rates.rate(home, currency));
        return new PriceView(p.getSku(), p.getName(), amount, currency);
    }
    ...
}
```

Every section below tests some part of this app, from the cheapest test to the most expensive.

12.2 The test pyramid in a Spring app

What it is

The **test pyramid** is a rule of thumb for how many tests of each kind to write. Write many small, fast tests at the bottom. Write fewer, slower tests that cover more of the system near the top. In a Spring app the layers map to how much of the container a test starts:

Layer	What starts	Spring annotation	Proves	Speed (this project)
Unit	nothing — `new` your class, mock (replace with a programmable fake) its collaborators	none (`@ExtendWith(MockitoExtension.class)`)	business logic, edge cases	~1 ms or less per test
Slice	one layer of the container (web, JPA, JSON, HTTP client)	`@WebMvcTest`, `@DataJpaTest`, `@JsonTest`, `@RestClientTest`, ...	mappings, validation, JSON shape, SQL, client parsing	context 0.03–0.14 s once warm (1.6 s for the very first, cold JVM — the Java Virtual Machine that runs the code, still loading classes)
Integration	the whole application context (mock servlet — a simulated Java web request/response layer — no port)	`@SpringBootTest` + `@AutoConfigureMockMvc`	the wiring: all real beans together	context ~0.7 s
Full stack / end-to-end	whole context + real server on a random port + real HTTP	`@SpringBootTest(webEnvironment = RANDOM_PORT)`	the app as a client sees it	context ~0.3–0.7 s, plus real network calls

The timings are the `Started <TestClass> in N seconds` lines Boot logs for each new context in this chapter's run:

```
... Started ProductRepositoryDataJpaTest in 1.574 seconds (process running for 2.205)   <- first context: cold JVM
... Started MockMvcFullContextTest in 0.726 seconds
... Started PriceViewJsonTest in 0.053 seconds
... Started ExchangeRateClientTest in 0.137 seconds
... Started ProductControllerWebMvcTest in 0.099 seconds
... Started PropertyPrecedenceTest in 0.034 seconds
... Started FullStackTest in 0.285 seconds
... Started RestTestClientMockMvcTest in 0.052 seconds
unit test: PriceView[sku=MUG-001, name=Mug, amount=10.85, currency=USD] in 0 ms
```

On a toy app the gap looks small. On a real service with 400 beans, Flyway migrations (versioned SQL scripts that update the database schema), Kafka (a message broker) and a security chain, a full context takes several seconds, and a suite of 300 test classes that each build one takes many minutes. The pyramid, together with context caching (§12.11), is what keeps a Spring build fast.

Why it matters

- **Fast feedback**: unit tests run in milliseconds, so you run them on every change.
- **Precise failures**: a failing unit or slice test points at one class. A failing end-to-end test only says "something is broken".
- **Confidence where it counts**: only an integration test catches a missing bean, a wrong property name, or two beans of the same type.

How to decide which layer

1. Is it pure logic (calculations, rules, mapping)? → **unit test**, no Spring.
2. Does it depend on something only Spring does (URL mapping, `@Valid`, JSON, a JPQL query, a `@Transactional` boundary)? → the **slice** for that layer.
3. Does it prove that beans work *together* (the real service with the real repository)? → `@SpringBootTest`, usually on the mock servlet environment.
4. Does it depend on a real server (filters in front of the servlet, HTTP compression, the port, the SSL setup — SSL/TLS is the encryption behind HTTPS)? → `RANDOM_PORT`.

Trade-offs

	Unit	Slice	`@SpringBootTest`	`RANDOM_PORT`
Speed	fastest	fast	slower	slowest
Realism	lowest (mocks everywhere)	real for one layer	real wiring	real HTTP
Failure points at	one class	one layer	"somewhere in the app"	"somewhere in the system"
Typical share of a suite	most	many	some	few

Common mistakes

- **Only** `@SpringBootTest`. Every test starts the whole app, and each small config difference forces a new context. This is the most common cause of 20-minute builds.
- **Mocking everything, even in integration tests**. If the repository, the client and the service are all mocks, the test proves nothing the unit test didn't.
- **Testing the framework**. You don't need a test showing that `JpaRepository.save` saves. Test *your* queries, mappings and rules.

Self-check

1. Which layer would you use to test that `POST /api/products` returns `400` for a bad SKU? → A `@WebMvcTest` slice: validation and error mapping are web-layer behaviour.
2. Which layer proves `cheaperThan` sorts by price? → `@DataJpaTest`: it runs the real JPQL on a real (embedded) database (one that runs inside the test's own JVM process, such as H2).
3. Why not test everything end-to-end? → Slow, and failures don't tell you where the bug is.

TL;DR

Most tests should be plain unit tests. Use a slice when the behaviour lives in Spring. Use a few `@SpringBootTest`s to prove the wiring. Use even fewer `RANDOM_PORT` tests.

12.3 Test dependencies in Boot 4

What changed

Boot 4 split the old monolithic (all-in-one) **auto-configuration** (Boot's beans that set things up automatically based on what is on the classpath) into **modules**, one per technology (Chapter 3). Test support was split the same way. Each technology now has a test starter named `spring-boot-starter-<technology>-test`. It pulls in the main starter, `spring-boot-starter-test`, and that technology's test module (slice annotation, test clients).

`testing-demo/pom.xml`:

```
<!-- Boot 4: one test starter per technology -->
<!-- @WebMvcTest, MockMvcTester, RestTestClient, JUnit 6, AssertJ, Mockito -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
<!-- @DataJpaTest, TestEntityManager -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa-test</artifactId>
  <scope>test</scope>
</dependency>
<!-- @RestClientTest, MockRestServiceServer -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-restclient-test</artifactId>
  <scope>test</scope>
</dependency>
```

Test starter	Adds (besides `spring-boot-starter-test`)
`spring-boot-starter-webmvc-test`	`@WebMvcTest`, `@AutoConfigureMockMvc`, `MockMvcTester`, `RestTestClient` / `TestRestTemplate` support
`spring-boot-starter-webflux-test`	`@WebFluxTest`, `WebTestClient` support, `reactor-test`
`spring-boot-starter-data-jpa-test`	`@DataJpaTest`, `TestEntityManager`, `@AutoConfigureTestDatabase`
`spring-boot-starter-jdbc-test`	`@JdbcTest`
`spring-boot-starter-data-r2dbc-test`	`@DataR2dbcTest`, `reactor-test`
`spring-boot-starter-restclient-test`	`@RestClientTest`, `MockRestServiceServer` wiring
`spring-boot-starter-jackson-test`	Jackson 3 + `spring-boot-starter-test` (`@JsonTest` itself lives in `spring-boot-test-autoconfigure`); pulled in by the webmvc/webflux/restclient test starters
`spring-boot-starter-security-test`	`spring-security-test` (`@WithMockUser`, `jwt()`, ...)
`spring-boot-starter-kafka-test`	`spring-kafka-test` (`@EmbeddedKafka`)

`spring-boot-starter-test` itself contains, in Boot 4.1.1: **JUnit Jupiter 6.0** (JUnit 6 needs Java 17+), **AssertJ**, **Mockito** (+ `mockito-junit-jupiter`), **Hamcrest** (an older matcher library), **JSONassert** (compares JSON documents), **JsonPath** (picks values out of JSON with paths like `$.amount`), **Awaitility** (wait for asynchronous results), XMLUnit, `spring-test` and `spring-boot-test`.

The slice annotations also **moved packages** with their modules. Your IDE (the code editor, such as IntelliJ IDEA) finds them, but old imports break when upgrading:

Annotation	Boot 3.x package	Boot 4 package
`@WebMvcTest`, `@AutoConfigureMockMvc`	`org.springframework.boot.test.autoconfigure.web.servlet`	`org.springframework.boot.webmvc.test.autoconfigure`
`@DataJpaTest`	`...boot.test.autoconfigure.orm.jpa`	`org.springframework.boot.data.jpa.test.autoconfigure`
`TestEntityManager`	`...boot.test.autoconfigure.orm.jpa`	`org.springframework.boot.jpa.test.autoconfigure`
`@RestClientTest`	`...boot.test.autoconfigure.web.client`	`org.springframework.boot.restclient.test.autoconfigure`
`@WebFluxTest`	`...boot.test.autoconfigure.web.reactive`	`org.springframework.boot.webflux.test.autoconfigure`
`@JsonTest`	`org.springframework.boot.test.autoconfigure.json`	unchanged
`@SpringBootTest`, `@TestConfiguration`	`org.springframework.boot.test.context`	unchanged

Common mistakes

- Adding only `spring-boot-starter-test` and wondering why `@WebMvcTest` doesn't resolve. In Boot 4 it lives in the **webmvc** test module.
- Keeping Boot 3 imports after an upgrade. Fix the imports as shown in the table above; the annotations work the same.

12.4 Unit tests: no Spring at all

What it is

A **unit test** creates the class with `new`, hands it **test doubles** (objects that stand in for real collaborators, see Java ch18), calls a method and checks the result. No application context, no annotations scanned, no database.

This works because the service uses **constructor injection** (Chapter 2: the class receives its dependencies as constructor parameters). The dependencies are ordinary constructor parameters, so the test passes them in itself.

```java
/** Layer 1 of the pyramid: a plain unit test. No Spring, no context, milliseconds. */
@ExtendWith(MockitoExtension.class)
class PriceServiceTest {

    @Mock ProductRepository products;
    @Mock ExchangeRateClient rates;
    PriceService service;

    @BeforeEach
    void setUp() {                       // constructor injection = no framework needed
        service = new PriceService(products, rates, "EUR");
    }

    @Test
    void convertsWithTheRemoteRate() {
        long t0 = System.nanoTime();
        when(products.findBySku("MUG-001"))
                .thenReturn(Optional.of(new Product("MUG-001", "Mug", new BigDecimal("10.00"))));
        when(rates.rate("EUR", "USD")).thenReturn(new BigDecimal("1.0850"));

        PriceView v = service.priceOf("MUG-001", "USD");

        System.out.println("unit test: " + v + " in " + (System.nanoTime() - t0) / 1_000_000 + " ms");
        assertThat(v.amount()).isEqualByComparingTo("10.85");   // 10.00 * 1.085 = 10.85
    }

    @Test
    void homeCurrencySkipsTheRemoteCall() {
        when(products.findBySku("MUG-001"))
                .thenReturn(Optional.of(new Product("MUG-001", "Mug", new BigDecimal("10.00"))));
        assertThat(service.priceOf("MUG-001", "EUR").amount()).isEqualByComparingTo("10.00");
        verify(rates, never()).rate(anyString(), anyString());
    }

    @Test
    void unknownSkuThrows() { ... assertThatThrownBy(...).hasMessage("No product with SKU NOPE"); }
}
```

Output:

```
unit test: PriceView[sku=MUG-001, name=Mug, amount=10.85, currency=USD] in 0 ms
```

What each part does:

- `@ExtendWith(MockitoExtension.class)` — Mockito's own JUnit extension creates the `@Mock` fields. This is not Spring.
- `isEqualByComparingTo("10.85")` — compares `BigDecimal`s by value. `isEqualTo` would fail on `10.85` vs `10.850` because `BigDecimal.equals` also compares the scale (the number of digits after the decimal point).
- `verify(rates, never())` — proves the rule "no remote call for the home currency". Only a unit test can check this so cheaply.

When not to use it

When the behaviour you care about is **produced by Spring**: a `@Transactional` rollback, a `@PreAuthorize` check, the JSON field names, a derived query. A unit test would bypass the proxy or the framework and prove nothing. Use a slice instead.

Common mistakes

- **Field injection** (`@Autowired private Repo repo;`) makes unit tests awkward: you need reflection (Java code that inspects and changes classes and private fields at runtime) or a Spring context just to set the field. Prefer constructors.
- **Starting Spring "because it's easier"**. `@SpringBootTest` to test a calculation costs seconds and hides design problems.

TL;DR

Constructor injection + Mockito + AssertJ = Spring-free unit tests that run in milliseconds. Most of your tests should look like this.

12.5 How the Spring TestContext Framework works

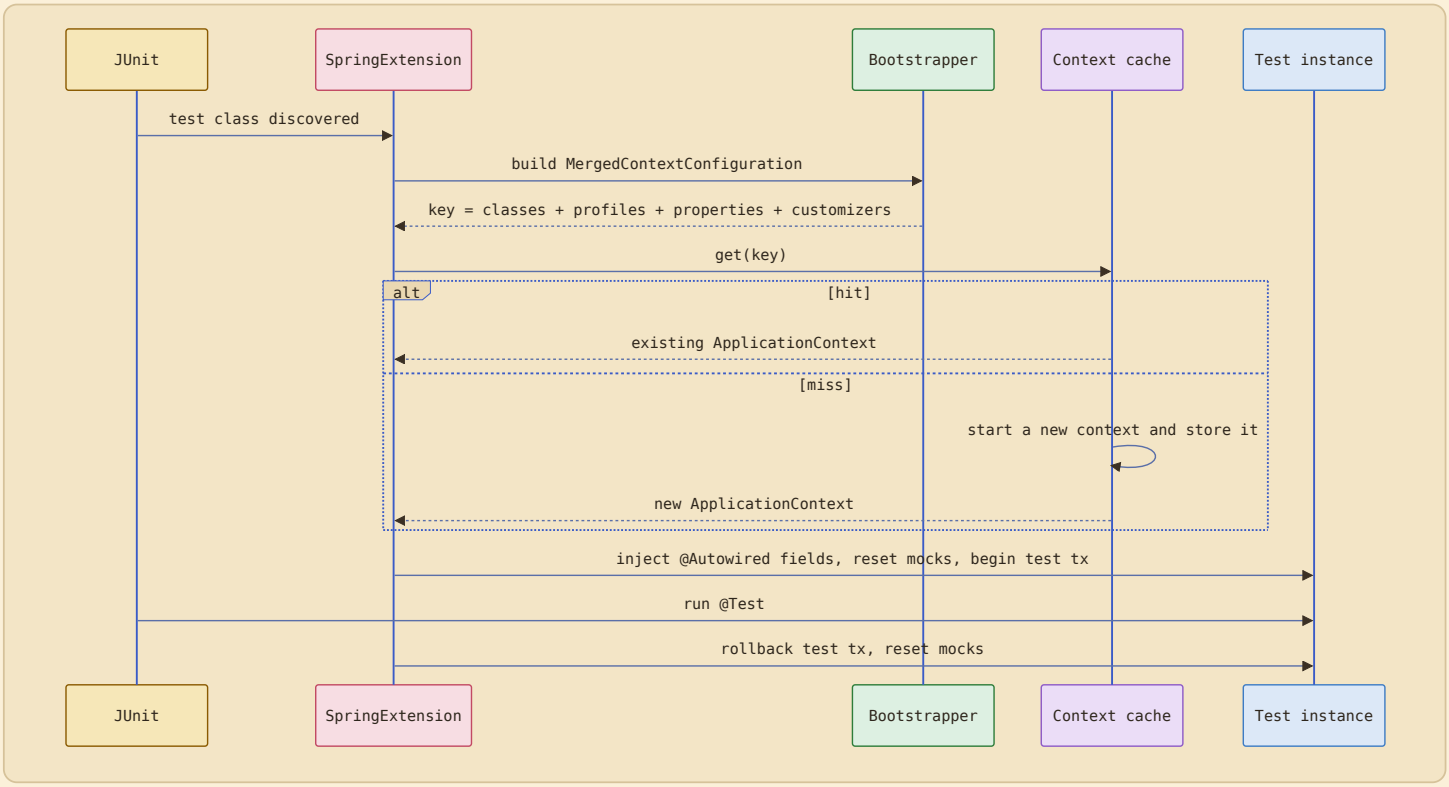
What it is

The **Spring TestContext Framework** (in `spring-test`) is the engine behind every Spring test annotation. Boot's `@SpringBootTest` and the slices are layers on top of it. Knowing its four moving parts explains most "why is my test slow / why is this bean missing" questions:

Part	What it is
`SpringExtension`	the JUnit Jupiter extension that connects JUnit's lifecycle to Spring. Every Boot test annotation is meta-annotated (itself annotated) with `@ExtendWith(SpringExtension.class)`, so you never add it yourself.
`TestContext`	per test class: holds the test class, current instance and method, and gives access to the `ApplicationContext`.
`TestExecutionListener`s	plug-ins called before/after each class and method: dependency injection into test fields, `@Transactional` test transactions, `@DirtiesContext`, `@MockitoBean` resets, `@Sql` scripts...
Context cache	a static (one copy per class, not per object), JVM-wide map from a *context configuration key* to a started `ApplicationContext`.

How it works

1. JUnit discovers the test class. `SpringExtension` asks a **bootstrapper** (Boot's `SpringBootTestContextBootstrapper`, or a slice's own) to build a `MergedContextConfiguration`: the configuration classes, active **profiles** (named sets of configuration, such as `test` or `prod`, switched on per run), test properties, context customizers (such as `@MockitoBean` definitions and `@DynamicPropertySource` methods), and so on.
2. For `@SpringBootTest`, when no classes are given, the bootstrapper searches **up the package tree** from the test's package for a class annotated with `@SpringBootConfiguration` (your `@SpringBootApplication`). This is why tests must live in the same package as the main class or below it.
3. The merged configuration is the **cache key**. If the cache has a context for it → reuse (**hit**). If not → create and start one (**miss**) and put it in the cache.
4. Listeners inject `@Autowired` fields into the test instance, start a transaction if the test is `@Transactional`, and so on.
5. After the test the listeners roll back the test transaction, reset `@MockitoBean` mocks, and close the context if it is marked dirty.
6. The context stays in the cache for later test classes with the same key, until the JVM exits (or it is evicted).



Self-check

1. Where does `@SpringBootTest` find the configuration if you give none? → It searches the test's package and its parents for `@SpringBootConfiguration`.
2. What decides whether two test classes share a context? → Whether their `MergedContextConfiguration`s are equal (the cache key).
3. Who injects `@Autowired` fields in a test? → A `TestExecutionListener` (`DependencyInjectionTestExecutionListener`), not the context itself — the test instance is not a bean.

12.6 `@SpringBootTest`: the whole application

What it is

`@SpringBootTest` starts your **entire application context** the way `SpringApplication.run` would: **component scanning** (finding classes annotated `@Component`, `@Service`, ... in your packages and turning them into beans), all auto-configuration, `application.properties`. It is the right tool for proving that beans work together.

The four web environments

webEnvironment	What you get	Use for
`MOCK` (default)	a web context with a **mock servlet environment** — no server, no port. Add `@AutoConfigureMockMvc` to get `MockMvcTester`.	most integration tests of a web app
`RANDOM_PORT`	a **real embedded server** (Tomcat/Netty, web servers started inside your app) on a free port; inject it with `@LocalServerPort`	end-to-end HTTP tests
`DEFINED_PORT`	a real server on `server.port` (default 8080)	rare: breaks parallel builds when the port is taken
`NONE`	no web environment at all	batch jobs (programs that process data in bulk and exit), messaging consumers, clients

> **Boot 4 change:** `@SpringBootTest` no longer auto-configures test clients by itself. Add `@AutoConfigureMockMvc` for MockMvc, `@AutoConfigureRestTestClient` for `RestTestClient`, `@AutoConfigureTestRestTemplate` for `TestRestTemplate`, `@AutoConfigureWebTestClient` for `WebTestClient` (per the Boot 4.0 migration guide).

Example: real beans, one fake

```
/** Whole context, mock servlet environment (no port): every real bean except the remote client. */
@SpringBootTest(properties = "catalog.home-currency=EUR")
@AutoConfigureMockMvc
class MockMvcFullContextTest {

    @TestConfiguration           // extra beans for this test only; not picked up by component scanning
    static class SeedData {
        @Bean ApplicationRunner seed(ProductRepository repo) {
            return args -> repo.save(new Product("LMP-001", "Lamp", new BigDecimal("40.00")));
        }
    }

    @Autowired MockMvcTester mvc;
    @Autowired ApplicationContext ctx;
    @MockitoBean ExchangeRateClient rates;    // the only fake: no network in this test

    @Test
    void realServiceAndDatabaseMockedRemote() throws Exception {
        given(rates.rate("EUR", "GBP")).willReturn(new BigDecimal("0.85"));

        MvcTestResult r = mvc.get().uri("/api/products/LMP-001/price?currency=GBP").exchange();

        System.out.println("@SpringBootTest beans: " + ctx.getBeanDefinitionCount());
        System.out.println("GET GBP -> " + r.getResponse().getContentAsString());
        assertThat(r).hasStatusOk().bodyJson().extractingPath("$.amount").isEqualTo(34.0);
    }
}
```

Output:

```
@SpringBootTest beans: 253
GET GBP -> {"sku":"LMP-001","name":"Lamp","amount":34.00,"currency":"GBP"}
```

What each part does:

- `properties = "..."` — adds test-only property values with high precedence (§12.15).
- **`@TestConfiguration`** — a `@Configuration` meant for tests. As a static nested class it is **added to** the application's configuration; it does not replace it. (A nested plain `@Configuration` would *replace* the auto-detected `@SpringBootApplication`, which is rarely what you want.) Top-level `@TestConfiguration` classes are skipped by component scanning and must be pulled in with `@Import`.
- The controller, service, repository, JPA and H2 are all real. Only the class that would call the internet is replaced.

When not to use it

- To test one controller's validation or one query: a slice is 5–10× faster and fails more precisely.
- With a different `properties=` / mock set on every class: each variation is a new context (§12.11).

Common mistakes

- **Expecting a `@Transactional` test to roll back what the server did in a `RANDOM_PORT` test.** The HTTP request runs on a server **thread** (an independent path of execution inside the JVM) in its own transaction, so the test's transaction can't roll it back. Clean up explicitly, as `FullStackTest` does with `repo.deleteAll()` in `@BeforeEach`.
- **Putting `@SpringBootTest` on a test in a package outside the main class's package** → "Unable to find a @SpringBootConfiguration".

Self-check

1. What's the default `webEnvironment`? → `MOCK`: no server, no port.
2. How do you add one extra bean for a single test class? → A static nested `@TestConfiguration` (or a top-level one + `@Import`).
3. How do you get `MockMvcTester` in a `@SpringBootTest`? → Add `@AutoConfigureMockMvc`.

12.7 Test slices

What it is

A **test slice** starts only the part of the application context needed for one layer. The slice annotation (for example `@WebMvcTest`) does three things:

1. It **disables full auto-configuration** and imports only the auto-configurations listed for that slice (for `@WebMvcTest`: MVC, Jackson — the JSON library that converts Java objects to JSON and back — validation, security filters, ...).
2. It adds a **type-exclude filter** to component scanning, so only matching beans of *your* code are registered (for `@WebMvcTest`: `@Controller`, `@ControllerAdvice`, `@JacksonComponent`, `Converter`, `Filter`, `WebMvcConfigurer`, `HandlerInterceptor`, ...). `@Service`, `@Repository` and `@Component` classes are **not** registered.
3. It adds test helpers (`MockMvcTester`, `TestEntityManager`, `MockRestServiceServer`, ...).

> **Analogy:** a slice is a stage rehearsal for one scene. Only the actors in that scene are on stage. Everyone else is replaced by a stand-in (`@MockitoBean`).

The slices you will meet

Annotation	Starts	Your beans registered	Extras	Transactional?
`@WebMvcTest`	Spring MVC, Jackson, validation, Security	controllers, advice, filters, converters, `WebMvcConfigurer`	`MockMvcTester`, `MockMvc`	no
`@WebFluxTest`	WebFlux, codecs, validation	controllers, `WebFluxConfigurer`, ...	`WebTestClient` (ch13)	no
`@DataJpaTest`	JPA, Hibernate (the JPA implementation), Spring Data JPA, embedded DB, Flyway/Liquibase	entities (classes mapped to tables), repositories	`TestEntityManager`, SQL logging on	**yes, rolled back**
`@JdbcTest`	`DataSource` (the factory for database connections), `JdbcTemplate`, `JdbcClient`, embedded DB	none	—	yes, rolled back
`@DataR2dbcTest`	R2DBC (non-blocking database access), Spring Data R2DBC	repositories	—	**no** (ch13)
`@JsonTest`	Jackson (or Gson / JSON-B)	`@JacksonComponent`, Jackson modules	`JacksonTester`, `BasicJsonTester`	no
`@RestClientTest`	`RestClient.Builder`, `RestTemplateBuilder`, Jackson	the classes you name	`MockRestServiceServer`	no
`@DataMongoTest`, `@DataRedisTest`, `@DataJdbcTest`, ...	that store	repositories	—	depends

`@WebMvcTest`

```
/** Web slice: only the MVC layer is real; the service is a Mockito mock. */
@WebMvcTest(ProductController.class)
class ProductControllerWebMvcTest {

    @Autowired MockMvcTester mvc;
    @Autowired ApplicationContext ctx;
    @MockitoBean PriceService service;       // replaces (or adds) the bean in the test context

    @Test
    void sliceContainsOnlyWebBeans() {
        System.out.println("@WebMvcTest beans: " + ctx.getBeanDefinitionCount()
                + ", ProductController present: " + (ctx.getBeanNamesForType(ProductController.class).length == 1)
                + ", ProductRepository present: " + (ctx.getBeanNamesForType(ProductRepository.class).length == 1));
        assertThat(ctx.getBeanNamesForType(ProductRepository.class)).isEmpty();
    }
    ...
}
```

Output:

```
@WebMvcTest beans: 127, ProductController present: true, ProductRepository present: false
```

127 beans instead of 253, and no JPA, no database, no `RestClient`. `@WebMvcTest(ProductController.class)` narrows it further to **one** controller. Without the argument, every controller in the app is registered, and each one's dependencies must then be mocked.

The web tests themselves are in §12.8.

`@DataJpaTest`

```
/** JPA slice: entities, repositories, an embedded DB; every test runs in a rolled-back transaction. */
@DataJpaTest
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class ProductRepositoryDataJpaTest {

    @Autowired ProductRepository repo;
    @Autowired TestEntityManager em;
    @Autowired DataSource dataSource;

    @Test @Order(1)
    void derivedAndJpqlQueries() throws Exception {
        System.out.println("DB = " + dataSource.getConnection().getMetaData().getURL()
                + ", test transaction active = " + TestTransaction.isActive());
        em.persist(new Product("MUG-001", "Mug", new BigDecimal("10.00")));
        em.persist(new Product("PEN-001", "Pen", new BigDecimal("2.50")));
        em.persist(new Product("LMP-001", "Lamp", new BigDecimal("40.00")));
        em.flush();   // send the INSERTs now
        em.clear();   // forget cached entities so the queries really hit the database

        assertThat(repo.findBySku("PEN-001")).map(Product::getName).contains("Pen");
        assertThat(repo.cheaperThan(new BigDecimal("15"))).extracting(Product::getSku)
                .containsExactly("PEN-001", "MUG-001");
        System.out.println("test 1 saw " + repo.count() + " rows");
    }

    @Test @Order(2)
    void previousTestWasRolledBack() {
        System.out.println("test 2 starts with " + repo.count() + " rows");
        assertThat(repo.count()).isZero();
    }
}
```

Output (`@DataJpaTest` turns on `spring.jpa.show-sql`, so Hibernate prints its SQL):

```
DB = jdbc:h2:mem:fa1ebe8d-78e8-4d82-a212-1a1b655ce2ab, test transaction active = true
Hibernate: select next value for product_seq
Hibernate: select next value for product_seq
Hibernate: insert into product (name,price,sku,id) values (?,?,?,?)
Hibernate: insert into product (name,price,sku,id) values (?,?,?,?)
Hibernate: insert into product (name,price,sku,id) values (?,?,?,?)
Hibernate: select p1_0.id,p1_0.name,p1_0.price,p1_0.sku from product p1_0 where p1_0.sku=?
Hibernate: select p1_0.id,p1_0.name,p1_0.price,p1_0.sku from product p1_0 where p1_0.price<=? order by p1_0.price
Hibernate: select count(*) from product p1_0
test 1 saw 3 rows
Hibernate: select count(*) from product p1_0
test 2 starts with 0 rows
```

What it shows:

- **`Embedded database`**: `@DataJpaTest` includes `@AutoConfigureTestDatabase`. Its default mode `Replace.NON_TEST` (since Boot 3.4) swaps your `DataSource` for a fresh embedded one (a random `jdbc:h2:mem:` name). It leaves the `DataSource` alone when it is already a *test* database, such as a Testcontainers `@ServiceConnection` (§12.14). Use `replace = NONE` to always keep the configured database.
- **`Rolled-back transaction`**: every test method runs in a transaction that is rolled back at the end, so test 2 sees 0 rows. That keeps tests independent without cleanup code.
- **`TestEntityManager`**: a test helper around the JPA `EntityManager` (`persist`, `persistAndFlush`, `find`, `clear`) for setting up data without going through the repository you are testing.
- **`flush()` + `clear()`**: without them the INSERTs would still sit in the **`persistence context`** (JPA's in-memory set of loaded and changed entities, also called the **`first-level cache`**) and `findBySku` might be answered from the first-level cache (ch08). Clearing forces real SQL.

> The rolled-back test transaction is convenient but can **`hide bugs`**. **`Lazy loading`** (fetching related rows only when first accessed) always works inside it, and nothing is ever **`committed`** (made permanent in the database). So constraint checks deferred to commit, `@TransactionalEventListener(AFTER_COMMIT)` and code that relies on its own transaction never run. Chapter 7 and Chapter 8 use non-transactional `@SpringBootTest`s for exactly this reason. Opt out per class with `@Transactional(propagation = Propagation.NOT_SUPPORTED)`.

`@JsonTest`

Checks the **`wire format`** (exactly how an object looks once **`serialized`**, i.e. turned into JSON text for sending): field names, number and date formats, and that nothing internal leaks.

```java
@JsonTest
class PriceViewJsonTest {

    @Autowired JacksonTester<PriceView> json;

    @Test
    void serialize() throws Exception {
        JsonContent<PriceView> out = json.write(new PriceView("MUG-001", "Mug", new BigDecimal("10.85"), "USD"));
        System.out.println("serialized: " + out.getJson());
        assertThat(out).extractingJsonPathStringValue("$.currency").isEqualTo("USD");
        assertThat(out).extractingJsonPathNumberValue("$.amount").isEqualTo(10.85);
        assertThat(out).doesNotHaveJsonPath("$.id");
    }

    @Test
    void deserialize() throws Exception {
        PriceView v = json.parseObject("""
                {"sku":"PEN-001","name":"Pen","amount":2.5,"currency":"EUR"}""");
        System.out.println("parsed: " + v);
        assertThat(v.amount()).isEqualByComparingTo("2.50");
    }
}
```

Output:

```
serialized: {"sku":"MUG-001","name":"Mug","amount":10.85,"currency":"USD"}
parsed: PriceView[sku=PEN-001, name=Pen, amount=2.5, currency=EUR]
```

The slice builds the same Jackson 3 `JsonMapper` your app uses, including `spring.jackson.*` properties and any `@JacksonComponent` classes. A plain `new JsonMapper()` in a unit test would miss those customisations.

`@RestClientTest`

Tests an HTTP **client** class without a network. The auto-configured `RestClient.Builder` is connected to a `MockRestServiceServer`, which checks what the client sent and plays back a canned (pre-recorded) answer.

```java
@RestClientTest(ExchangeRateClient.class)
@TestPropertySource(properties = "rates.base-url=https://rates.test")
class ExchangeRateClientTest {

    @Autowired ExchangeRateClient client;
    @Autowired MockRestServiceServer server;

    @Test
    void parsesTheRate() {
        server.expect(requestTo("https://rates.test/rates?from=EUR&to=USD"))
              .andExpect(method(HttpMethod.GET))
              .andRespond(withSuccess("""
                      {"from":"EUR","to":"USD","rate":1.0850}""", MediaType.APPLICATION_JSON));

        var rate = client.rate("EUR", "USD");
        System.out.println("rate EUR->USD = " + rate);
        assertThat(rate).isEqualByComparingTo("1.085");
        server.verify();                  // every expected request really happened
    }

    @Test
    void serverErrorBecomesAnException() {
        server.expect(requestTo("https://rates.test/rates?from=EUR&to=JPY"))
              .andRespond(withServerError());

        assertThatThrownBy(() -> client.rate("EUR", "JPY"))
                .isInstanceOf(HttpServerErrorException.InternalServerError.class);
        System.out.println("500 from rates API -> HttpServerErrorException.InternalServerError");
    }
}
```

Output:

```
rate EUR->USD = 1.0850
500 from rates API -> HttpServerErrorException.InternalServerError
```

`MockRestServiceServer` works only for clients built from the auto-configured builder. More client-testing options (a real local server, WireMock — a configurable fake HTTP server) are in Chapter 6.

Common mistakes

- **"No qualifying bean of type PriceService" in a `@WebMvcTest`.** Services are not part of the slice. Add `@MockitoBean PriceService`, or `@Import` the real class if it is cheap.
- **Extra `@ComponentScan` or `@EnableJpaRepositories` on the main class.** The slices rely on Boot's type-exclude filters. A hand-written `@ComponentScan` on `@SpringBootApplication` bypasses them, and suddenly your `@WebMvcTest` loads repositories. Put such configuration in a separate `@Configuration` class.
- **Configuration beans the slice doesn't know about.** A `@Configuration` class in your main package *is* scanned by every slice. If it creates a heavy bean (a Kafka client), every slice pays for it. Guard it with a profile or a condition, or move it out of the scanned package.
- **Assuming `@DataJpaTest` = production database behaviour.** H2 accepts SQL that PostgreSQL rejects and vice versa. Test database-specific SQL against the real engine (§12.14).

When not to use it

Don't write a slice for every class. If a controller only forwards to a service, one web-slice test per endpoint (happy path — the normal, everything-valid case — + error mapping) is enough.

In the interview

Q: What is a test slice and why use one? 30-second answer: "A slice like `@WebMvcTest` or `@DataJpaTest` starts only one layer of the application context: its auto-configurations plus the matching beans from my code. It's much faster than `@SpringBootTest` and tests exactly what Spring adds to that layer: mappings, validation and JSON for the web, real SQL for JPA. Everything outside the slice I mock with `@MockitoBean`."

- *Does `@WebMvcTest` load `@Service` beans?* → No; only web components. Mock or `@Import` services.
- *Are `@DataJpaTest` tests transactional?* → Yes, each test rolls back; and the `DataSource` is replaced by an embedded one unless it's already a test database or you set `replace = NONE`.
- *How would you test a `RestClient`-based client?* → `@RestClientTest` + `MockRestServiceServer`, or a real local stub server for timeouts and connection-level behaviour.

Self-check

1. Why is `ProductRepository` absent in `@WebMvcTest`? → The slice's type-exclude filter only registers web components; JPA auto-configuration isn't imported.
2. Why call `em.flush(); em.clear();` before querying? → To send the INSERTs and empty the first-level cache so the query really hits the database.
3. What does `server.verify()` check? → That every expected request was actually made.

TL;DR

Slices = partial contexts: `@WebMvcTest` (web), `@DataJpaTest` (JPA, rolled back, embedded DB), `@JsonTest` (Jackson), `@RestClientTest` (clients + `MockRestServiceServer`). Mock what's outside the slice.

12.8 `MockMvcTester`: AssertJ-style MockMvc

What it is

MockMvc sends requests straight into the `DispatcherServlet` (Spring MVC's single front controller that receives every request and routes it to your controller) without a real server or network. The request goes through filters, mapping, argument resolution (turning request parts into method parameters), validation, the controller, exception handlers and message converters (which turn objects into JSON and back), exactly as in production.

`MockMvcTester` (Framework 6.2, auto-configured since Boot 3.4) wraps MockMvc in **AssertJ** assertions. You build a request, call `exchange()`, and assert on the result with `assertThat(...)`. It replaces the older `perform(...).andExpect(...)` style.

	Classic `MockMvc`	`MockMvcTester`
Request	`mvc.perform(get("/x"))`	`mvc.get().uri("/x").exchange()`
Assertions	`andExpect(status().isOk())`, `jsonPath(...).value(...)` (Hamcrest)	`assertThat(result).hasStatusOk()`, `.bodyJson().extractingPath("$.x")` (AssertJ)
Checked exceptions (ones Java forces you to catch or declare)	`perform` throws `Exception`	none
Unhandled exceptions	thrown out of `perform`	captured in the result: `assertThat(result).hasFailed()`
Both available?	yes — Boot auto-configures both with `@WebMvcTest` / `@AutoConfigureMockMvc`	

Example

```
@Test
void returnsPriceAsJson() throws Exception {
    given(service.priceOf("MUG-001", "USD"))
            .willReturn(new PriceView("MUG-001", "Mug", new BigDecimal("10.85"), "USD"));

    MvcTestResult r = mvc.get().uri("/api/products/{sku}/price?currency=USD", "MUG-001").exchange();

    System.out.println("GET price -> " + r.getResponse().getStatus() + " " + r.getResponse().getContentAsString());
    assertThat(r).hasStatusOk()
            .hasContentType(MediaType.APPLICATION_JSON)
            .bodyJson().extractingPath("$.amount").isEqualTo(10.85);
}

@Test
void unknownSkuIsProblemDetail404() throws Exception {
    given(service.priceOf("NOPE-000", "EUR")).willThrow(new ProductNotFoundException("NOPE-000"));

    MvcTestResult r = mvc.get().uri("/api/products/NOPE-000/price").exchange();

    assertThat(r).hasStatus(404)
            .hasContentType(MediaType.APPLICATION_PROBLEM_JSON)
            .bodyJson().extractingPath("$.detail").isEqualTo("No product with SKU NOPE-000");
}

@Test
void invalidBodyIs400AndServiceIsNeverCalled() {
    MvcTestResult r = mvc.post().uri("/api/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                    {"sku":"mug1","name":"","price":0}""")
            .exchange();

    assertThat(r).hasStatus(400);
    verifyNoInteractions(service);
}

@Test
void validBodyIs201WithLocation() {
    given(service.create(any())).willReturn(new Product("MUG-002", "Big mug", new BigDecimal("12.50")));
    MvcTestResult r = mvc.post().uri("/api/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                    {"sku":"MUG-002","name":"Big mug","price":12.50}""")
            .exchange();
    assertThat(r).hasStatus(201).hasHeader("Location", "/api/products/MUG-002/price");
}
```

Output:

```
GET unknown -> 404 {"detail":"No product with SKU NOPE-000","instance":"/api/products/NOPE-000/price","status":404,"title":"Not
Found"}
GET price -> 200 {"sku":"MUG-001","name":"Mug","amount":10.85,"currency":"USD"}
POST valid -> 201 Location=/api/products/MUG-002/price
... WARN ... DefaultHandlerExceptionResolver : Resolved [org.springframework.web.bind.MethodArgumentNotValidException: Validation
failed for argument [0] ... with 3 errors: ...]
POST invalid -> 400
```

What the tests prove:

- The URL template, the `currency` default and the JSON field names (`amount`, not `price`).
- `ProductNotFoundException` → **`application/problem+json`** with status, title, detail and instance (RFC 9457, ch05).
- Invalid input is rejected **before** the controller runs: `verifyNoInteractions(service)`. The WARN line is Spring logging the three constraint violations (bad pattern, blank name, price below 0.01).
- A `201 Created` with a `Location` header for the new resource.

Common mistakes

- **Asserting on `getContentAsString()` with `equals`.** Key order and spacing make string comparisons brittle. Use `bodyJson()` paths, or `bodyJson().isLenientlyEqualTo("{...}")`.
- **Expecting servlet-container behaviour.** MockMvc has no real Tomcat. Multipart size limits, HTTP/2, compression and error pages rendered by the container are not exercised. Use `RANDOM_PORT` for those.
- **Forgetting security.** With Spring Security on the **classpath** (the set of JARs and folders the JVM loads classes from), `@WebMvcTest` applies Boot's default security, not your `SecurityFilterChain`, unless you `@Import` your config (ch10).

Self-check

1. What runs in a MockMvc request? → The real `DispatcherServlet` pipeline (filters, mapping, conversion, validation, advice) without a network or server.
2. What does `bodyJson().extractingPath("$.amount")` use under the hood? → JsonPath on the response body, with AssertJ assertions on the extracted value.
3. How do you assert an unhandled exception with `MockMvcTester`? → `assertThat(result).hasFailed().failure().isInstanceOf(...)`.

12.9 `RestTestClient` (Framework 7.0)

What it is

`RestTestClient` is a new test client in **Spring Framework 7.0**. It has the **fluent** (method calls chained one after another) `WebTestClient`-style API (`get().uri(...).exchange().expectStatus()...`), but for **servlet** (Spring MVC) applications and without needing WebFlux. One API can target different back ends:

Binding	Request goes to	Typical use
`RestTestClient.bindTo(mockMvc)`	an existing `MockMvc` (no server)	slice / mock-environment tests
`bindToController(controller)`	a standalone MockMvc for one controller	pure controller tests without Spring Boot
`bindToRouterFunction(...)`	functional MVC routes	`RouterFunction` tests
`bindToApplicationContext(ctx)`	MockMvc built from a `WebApplicationContext`	custom setups
`bindToServer()`	a **real HTTP server**	`RANDOM_PORT` end-to-end tests

So the same test code can run against MockMvc or against a real server.

Boot 4 status (verified): Boot 4.0 added auto-configuration for it. Annotate the test class with `@AutoConfigureRestTestClient` (module `spring-boot-resttestclient`, included in `spring-boot-starter-webmvc-test`). In a mock environment the client is bound to the `MockMvc`. In a `RANDOM_PORT`/`DEFINED_PORT` test it points at the running server, with the base URL already set. Without the annotation there is no bean. Removing it from the test below fails with:

```
No qualifying bean of type 'org.springframework.test.web.servlet.client.RestTestClient' available
```

Example against MockMvc

```
/** RestTestClient (Framework 7.0) bound to MockMvc: a fluent HTTP-client API, no real server. */
@WebMvcTest(ProductController.class)
@AutoConfigureRestTestClient
class RestTestClientMockMvcTest {

    @Autowired RestTestClient client;
    @MockitoBean PriceService service;

    @Test
    void fluentExpectations() {
        given(service.priceOf("MUG-001", "EUR"))
                .willReturn(new PriceView("MUG-001", "Mug", new BigDecimal("10.00"), "EUR"));

        PriceView body = client.get().uri("/api/products/MUG-001/price")
                .exchange()
                .expectStatus().isOk()
                .expectHeader().contentType("application/json")
                .expectBody(PriceView.class)              // JSON -> record, with the app's JsonMapper
                .returnResult().getResponseBody();

        System.out.println("RestTestClient (MockMvc) body = " + body);
        assertThat(body.amount()).isEqualByComparingTo("10.00");
    }

    @Test
    void assertjStyle() {
        given(service.priceOf("NOPE-000", "EUR")).willThrow(new ProductNotFoundException("NOPE-000"));
        RestTestClient.ResponseSpec spec = client.get().uri("/api/products/NOPE-000/price").exchange();

        assertThat(RestTestClientResponse.from(spec))       // AssertJ adapter (Framework 7.0)
                .hasStatus(404)
                .bodyJson().extractingPath("$.title").isEqualTo("Not Found");
        System.out.println("RestTestClient (AssertJ) 404 title = Not Found");
    }
}
```

Output:

```
RestTestClient (AssertJ) 404 title = Not Found
RestTestClient (MockMvc) body = PriceView[sku=MUG-001, name=Mug, amount=10.00, currency=EUR]
```

The real-server version is in §12.13.

Which test client?

Client	Stack	Server needed	Status in Boot 4
`MockMvcTester`	MVC	no	recommended for MVC slice/mock tests
`RestTestClient`	MVC (any HTTP server with `bindToServer`)	optional	**new**, recommended for fluent HTTP-style tests
`WebTestClient`	WebFlux (can also bind to MockMvc)	optional	recommended for WebFlux (ch13)
`TestRestTemplate`	any	yes	still available (`@AutoConfigureTestRestTemplate`, moved to `org.springframework.boot.resttestclient`); `RestTemplate` itself is heading for deprecation (ch06)

Self-check

1. What does `RestTestClient` add over `MockMvcTester`? → A client-style API that can target MockMvc *or* a real server with the same test code, and body decoding to types (`expectBody(PriceView.class)`).
2. What annotation do you need in Boot 4 to inject it? → `@AutoConfigureRestTestClient`.

12.10 `@MockitoBean` and `@MockitoSpyBean`

What it is

- `@MockitoBean` (Framework 6.2, in `org.springframework.test.context.bean.override.mockito`) puts a **Mockito mock into the application context**. It replaces the bean of that type (or name), or adds one if none exists. Every bean that

depends on it, such as the controller, receives the mock.

- `@MockitoSpyBean` wraps the **real existing bean** in a Mockito **spy**. Real methods run, but you can `verify` calls or **stub** single methods (tell them what to return).

Compare with Mockito's own `@Mock`: `@Mock` creates a mock that only your test holds. `@MockitoBean` creates a mock that **the container injects** into other beans.

> **Boot 4:** Boot's `@MockBean` and `@SpyBean` (deprecated in 3.4) are **removed**. Replace them with `@MockitoBean` / `@MockitoSpyBean`. One difference: the new annotations work on test-class fields (including superclasses and `@Nested` classes), **not inside** `@Configuration` **classes**. To share mocks, declare them on a common base class, or at class level with `@MockitoBean(types = {A.class, B.class})`.

How it works

1. While building the `MergedContextConfiguration`, Spring finds bean-override fields and adds them to the **cache key** (so each distinct set of mocks is a different context, see §12.11).
2. When the context starts, the bean definition for that type is replaced by one that returns the Mockito mock (or, for a spy, the real bean is wrapped after creation).
3. The mock is injected into the test field.
4. **After each test method the mock is reset** (default `MockReset.AFTER`), so stubbing from one test doesn't leak into the next.

Example: a spy on the real client

In `FullStackTest` the `ExchangeRateClient` is real and makes real HTTP calls. The spy lets the test count them:

```
@MockitoSpyBean ExchangeRateClient rates; // the REAL bean, wrapped so we can verify calls
...
verify(rates, times(1)).rate("EUR", "USD");   // EUR request did not call the API
```

The mocks in the `@WebMvcTest`s above (`@MockitoBean PriceService service`) are the more common case.

Related: `@TestBean`

`@TestBean` (Framework 6.2) overrides a bean with an instance returned by a **static factory method** (a `static` method whose job is to create and return an object) in the test class. Use it when a hand-written fake is clearer than a Mockito mock:

```
@TestBean ExchangeRateClient rates;                    // looks for static method rates()
static ExchangeRateClient rates() { return new FixedRateClient(new BigDecimal("1.10")); }
```

Common mistakes

- **Mocking in a way that changes the context for every class.** Class A mocks `PriceService`, class B mocks `ExchangeRateClient`, class C mocks both: that's three contexts. Standardise the mock set in a base class.
- **Mocking what you're trying to test.** A `@SpringBootTest` with the repository, the client *and* the service mocked is a slow unit test.
- `@MockitoSpyBean` **on a bean that doesn't exist** → the context fails to start. A spy needs a real bean to wrap.
- **Final classes/methods.** Mockito 5 uses the inline mock maker by default, so finals can be mocked. On current JDKs it attaches itself as an **agent** (a helper that can rewrite classes while the JVM runs) at runtime and prints *"Mockito is currently self-attaching..."*. The Mockito docs recommend adding it as a `-javaagent` in the Surefire (Maven's unit-test runner plugin) configuration before the JDK disallows dynamic agent loading by default.

In the interview

Q: `@Mock` **vs** `@MockitoBean`**?** → `@Mock` is plain Mockito: a mock only the test holds, no Spring. `@MockitoBean` registers the mock *in the application context*, so Spring injects it into other beans. It also becomes part of the context cache key.

- *What replaced `@MockBean` in Boot 4?* → `@MockitoBean` from Spring Framework (6.2+); `@MockBean` was removed.
- *When would you use a spy bean?* → To keep real behaviour but verify interactions or stub one method, e.g. count calls to a real client.

TL;DR

`@MockitoBean` = mock inside the context. `@MockitoSpyBean` = spy around the real bean. Both are reset after each test and both change the cache key.

12.11 Test context caching

What it is

Starting an application context is the most expensive part of a Spring test. The TestContext Framework therefore keeps started contexts in a **static cache** that lives for the whole test JVM. The second test class with an *identical* configuration gets the already-running context in about 0 ms.

> **Analogy:** a restaurant kitchen that keeps the oven hot between orders. Any order that needs the same oven setting is cooked at once. A different setting means heating up another oven.

What is in the cache key

A new context is created when **any** of these differ between two test classes:

- configuration classes / locations (`@SpringBootTest(classes=...)`, `@ContextConfiguration`, `@Import` on the test, nested `@TestConfiguration`)
- active profiles (`@ActiveProfiles`)
- test properties (`@TestPropertySource`, `@SpringBootTest(properties=...)`) and property source files
- **context customizers**: `@MockitoBean` / `@MockitoSpyBean` / `@TestBean` definitions, `@DynamicPropertySource` methods, `@AutoConfigureMockMvc` and other `@AutoConfigure...` settings, `webEnvironment`
- the slice type (`@WebMvcTest(A.class)` and `@WebMvcTest(B.class)` are different)
- context initializers, the context loader, the parent context (`@ContextHierarchy`)

`@DirtiesContext` doesn't change the key. It **closes and evicts** the context after the class or method, so the next user must build it again.

The measured demo

`context-caching` has one base class and seven test classes, run in name order (`junit.jupiter.testclass.order.default=...ClassOrderer$ClassName` in `junit-platform.properties`). To make startup cost visible, the app has a bean that sleeps **400 ms** in its constructor, standing in for real startup work such as warming a cache or loading a model:

```
@Component
public static class SlowStartup {
    public static final AtomicInteger CREATED = new AtomicInteger();
    public SlowStartup() throws InterruptedException {
        Thread.sleep(400);
        CREATED.incrementAndGet();          // = number of contexts built so far
    }
}
```

```
/** One shared configuration: every subclass that adds nothing reuses the same cached context. */
@SpringBootTest
@AutoConfigureMockMvc
abstract class BaseIntegrationTest { }

class C1_GreetingApiTest   extends BaseIntegrationTest { ... }  // first user -> MISS
class C2_SameConfigTest    extends BaseIntegrationTest { ... }  // identical config -> HIT
class C3_WithMockTest      extends BaseIntegrationTest {        // a @MockitoBean changes the key -> MISS
    @MockitoBean CachingApplication.GreetingService service; ... }
@TestPropertySource(properties = "feature.discounts=true")    // different properties -> MISS
class C4_OtherPropertyTest extends BaseIntegrationTest { ... }
@DirtiesContext                                               // HIT, then closed and evicted
class C5_DirtiesContextTest extends BaseIntegrationTest { ... }
class C6_AfterDirtiesTest  extends BaseIntegrationTest { ... }  // base config again, evicted -> MISS
class C7_BackToBaseTest    extends BaseIntegrationTest { ... }  // HIT on the rebuilt context
```

A small `TestExecutionListener` measures, for each class, the time from `beforeTestClass` until the context is available, and reads the cache statistics. It is registered for all tests in `src/test/resources/META-INF/spring.factories`:

```java
public class CacheReportListener implements TestExecutionListener, Ordered {

    /** The JVM-wide context cache is reachable through this protected accessor. */
    static final class Stats extends DefaultCacheAwareContextLoaderDelegate {
        static ContextCache cache() { return new Stats().getContextCache(); }
    }

    @Override public int getOrder() { return Ordered.LOWEST_PRECEDENCE; }
    @Override public void beforeTestClass(TestContext tc) { start = System.nanoTime(); }

    @Override
    public void prepareTestInstance(TestContext tc) {
        ...
        tc.getApplicationContext();                         // loads it, or takes it from the cache
        long ms = (System.nanoTime() - start) / 1_000_000;
        ContextCache c = Stats.cache();
        System.out.printf("%-26s context in %4d ms | cache size=%d hits=%d misses=%d | contexts built=%d%n", ...);
    }
}
```

Output:

```
C1_GreetingApiTest         context in  892 ms | cache size=1 hits=3 misses=1 | contexts built=1
C2_SameConfigTest          context in    0 ms | cache size=1 hits=18 misses=1 | contexts built=1
C3_WithMockTest            context in  575 ms | cache size=2 hits=32 misses=2 | contexts built=2
C4_OtherPropertyTest       context in  503 ms | cache size=3 hits=45 misses=3 | contexts built=3
C5_DirtiesContextTest      context in    0 ms | cache size=3 hits=60 misses=3 | contexts built=3
C6_AfterDirtiesTest        context in  467 ms | cache size=3 hits=73 misses=4 | contexts built=4
C7_BackToBaseTest          context in    0 ms | cache size=3 hits=88 misses=4 | contexts built=4
```

How to read it:

- **misses** = contexts actually built (4); **contexts built** (the counter in `SlowStartup`) agrees.
- **hits** count every *lookup* of the cache, not test classes. Spring's own listeners call `getApplicationContext()` many times per test, so the number climbs quickly. Look at how it changes, not at the absolute value.
- Every miss costs ~0.4 s of simulated startup plus ~0.1 s of real Boot work; the first one also pays for JVM/class loading (892 ms). Every hit costs **0 ms**.
- A mock (C3) and one property (C4) each forced a whole new context. `@DirtiesContext` (C5) threw away a good context, so C6 had to rebuild it.
- Seven classes, four builds. With one shared configuration and no `@DirtiesContext`, it would be **one** build.

To see Spring's own statistics, set the logger `org.springframework.test.context.cache` to `DEBUG`. It logs lines like `Spring test ApplicationContext cache statistics: [DefaultContextCache@... size = 1, maxSize = 32, ... hitCount = ..., missCount = ...]`.

Limits and Framework 7 behaviour

Setting	Default	How to change
Maximum cached contexts	**32**, least-recently-used evicted (and closed)	JVM system property or `spring.properties`: `spring.test.context.cache.maxSize=16`
Context pausing (new in Framework 7.0)	`ON_CONTEXT_SWITCH`: when the next test needs a *different* context, the inactive one is **paused**, meaning its `Lifecycle` / `SmartLifecycle` beans (beans with start/stop methods: schedulers, message listener containers) are stopped, and restarted when it is used again	`spring.test.context.cache.pause=always` or `never`

Pausing fixes an old problem: a cached context with a `@Scheduled` job or a Kafka/JMS (Java Message Service, the standard Java messaging API) listener kept consuming messages while *another* test class was running, stealing its messages or skewing its results.

Tips for a fast suite

1. Put the common annotations (`@SpringBootTest`, `@AutoConfigureMockMvc`, `@ActiveProfiles("test")`, the shared `@MockitoBean`s, container config) on **one abstract base class** and extend it.
2. Prefer **slices** for layer tests; they're small, so their extra contexts are cheap.
3. Avoid `@DirtiesContext`. Reset state instead (clean tables, `reset()` fakes). If you must, use it on the few tests that truly break the context.
4. Watch for **one-off** `properties=` values and one-off mocks. Each one is a new context that stays in memory until eviction.
5. Surefire runs all classes in one JVM by default (`forkCount=1`, `reuseForks=true`). That's what makes the cache useful. Settings that fork (start a separate) JVM per class throw the cache away.

Common mistakes

- **Too many contexts in memory.** Up to 32 full contexts, each with its thread pools and connection pools (sets of reusable threads and open database connections) and embedded servers, can exhaust memory, database connections or ports. Symptoms are `Too many connections` errors or out-of-memory errors late in the build.
- **Static state across contexts.** Two contexts in the same JVM share static fields, and the same H2 database name if the URL is fixed.

In the interview

Q: Why is our Spring test suite slow and how do you fix it? "Almost always too many application contexts. The TestContext Framework caches contexts by their configuration. Every different set of `@MockitoBean`s, properties, profiles or `@Import`s, and every `@DirtiesContext`, builds a new one. I'd enable DEBUG logging for `org.springframework.test.context.cache` to count misses, move shared configuration to a base class, replace one-off `@SpringBootTest`s with slices or unit tests, and remove `@DirtiesContext` where cleanup is enough."

- *What's the default cache size?* → 32, LRU eviction; `spring.test.context.cache.maxSize`.
- *What is context pausing?* → Framework 7.0: inactive cached contexts get their lifecycle beans stopped when the tests switch to another context, and restarted on reuse.

Self-check

1. Two classes differ only in `@MockitoBean`. Same context? → No, bean overrides are part of the key.
2. What does `@DirtiesContext` do to the cache? → Closes the context and removes it; the next class with that key rebuilds it.
3. Why does `hits` grow by ~15 per class in the demo? → It counts every cache lookup, and several listeners look up the context for each test.

TL;DR

Same configuration → same (cached) context. Mocks, properties, profiles, imports and `@DirtiesContext` cost a new one. Aim for a handful of configurations per suite.

12.12 `@DynamicPropertySource`: properties known only at runtime

The problem it solves

Some configuration only exists **after** the test starts: a stub server's random port, a container's mapped port, a temporary directory. You cannot write it in `application-test.properties`.

What it is

A **static** method annotated with `@DynamicPropertySource` (Framework 5.2.5) that receives a `DynamicPropertyRegistry`. Each property is registered with a `Supplier` (a Java function that takes nothing and returns a value), which is called lazily (only when needed) when the property is first read, so it can return a value that is only known once something has started. These properties have the **highest precedence**, above `application.properties`, `@TestPropertySource` and `@SpringBootTest(properties)`.

Framework 6.2 added `DynamicPropertyRegistrar`: a *bean* that registers properties. Use it when the value comes from another bean in the context (e.g. a stub server defined as a bean) rather than from a static field.

Example: a fake exchange-rate API on a random port

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureRestTestClient
@ActiveProfiles("test")
class FullStackTest {

    // A tiny fake "rates" API started before the Spring context (JDK built-in HTTP server).
    static final HttpServer RATES = startRatesServer();

    static HttpServer startRatesServer() {
        try {
            HttpServer s = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0); // port 0 = any free port
            s.createContext("/rates", ex -> {
                byte[] body = """
                        {"from":"EUR","to":"USD","rate":1.10}""".getBytes(StandardCharsets.UTF_8);
                ex.getResponseHeaders().add("Content-Type", "application/json");
                ex.sendResponseHeaders(200, body.length);
                ex.getResponseBody().write(body);
                ex.close();
            });
            s.start();
            return s;
        } catch (IOException e) { throw new IllegalStateException(e); }
    }

    @DynamicPropertySource
    static void ratesUrl(DynamicPropertyRegistry registry) {
        // evaluated lazily, after the fake server has picked its port
        registry.add("rates.base-url", () -> "http://127.0.0.1:" + RATES.getAddress().getPort());
    }

    @AfterAll
    static void stop() { RATES.stop(0); }
    ...
}
```

The `ExchangeRateClient` bean is created with `rates.base-url` pointing at the fake server, so the test exercises the real `RestClient`, real JSON parsing and real **sockets** (the operating system's network connections). The server is the JDK's built-in `com.sun.net.httpserver.HttpServer`, so no extra dependency is needed.

Common mistakes

- **Non-static method** → the annotation is ignored or rejected; it must be `static` because it runs before any test instance exists.
- **Passing the value, not a supplier** (`registry.add("x", port)` computed too early). Pass a lambda (a short inline function such as `() -> port`).
- **Every class with its own `@DynamicPropertySource`** → each is a different cache key. Put shared containers/stubs and their properties in a base class.

Self-check

1. Why a `Supplier` and not a value? → The value may not exist yet when the method runs; the supplier is read when the property is first needed.
2. What wins: `@TestPropertySource` or `@DynamicPropertySource`? → `@DynamicPropertySource` (highest precedence).

12.13 Full-stack tests on a real port

`FullStackTest` (continued) runs the **whole app on a real Tomcat port**, with a real H2 database and a real HTTP call to the fake rates API, using `RestTestClient` bound to the server:

```
    @LocalServerPort int port;
    @Autowired RestTestClient client;        // bound to the running server in RANDOM_PORT tests
    @Autowired ProductRepository repo;
    @MockitoSpyBean ExchangeRateClient rates; // the REAL bean, wrapped so we can verify calls

    @BeforeEach
    void data() {
        repo.deleteAll();
        repo.save(new Product("MUG-001", "Mug", new BigDecimal("10.00")));
    }

    @Test
    void endToEnd() {
        System.out.println("app on port " + port + ", rates API on port " + RATES.getAddress().getPort());

        PriceView usd = client.get().uri("/api/products/MUG-001/price?currency=USD")
                .exchange().expectStatus().isOk()
                .expectBody(PriceView.class).returnResult().getResponseBody();
        PriceView eur = client.get().uri("/api/products/MUG-001/price")
                .exchange().expectStatus().isOk()
                .expectBody(PriceView.class).returnResult().getResponseBody();

        System.out.println("USD -> " + usd + "\nEUR -> " + eur);
        assertThat(usd.amount()).isEqualByComparingTo("11.00");
        verify(rates, times(1)).rate("EUR", "USD");   // EUR request did not call the API
    }

    @Test
    void createThenRead() {
        client.post().uri("/api/products")
                .contentType(MediaType.APPLICATION_JSON)
                .body("""
                        {"sku":"PEN-001","name":"Pen","price":2.50}""")
                .exchange()
                .expectStatus().isCreated()
                .expectHeader().location("/api/products/PEN-001/price");

        assertThat(repo.findBySku("PEN-001")).isPresent();   // committed for real, no test rollback
        System.out.println("rows after POST = " + repo.count());
    }
```

Output:

```
app on port 51807, rates API on port 51806
USD -> PriceView[sku=MUG-001, name=Mug, amount=11.00, currency=USD]
EUR -> PriceView[sku=MUG-001, name=Mug, amount=10.00, currency=EUR]
rows after POST = 2
```

Notes:

- `11.00` = 10.00 × 1.10 from the fake server: the full chain controller → service → repository → H2, and service → `RestClient` → HTTP → JSON, worked.
- The spy confirms the business rule end-to-end: exactly **one** remote call, for USD only.
- `rows after POST = 2` (MUG-001 from `@BeforeEach` + the new PEN-001): data is **committed**. Hence `repo.deleteAll()` before each test instead of relying on rollback.
- Ports are random on every run, so parallel builds on one CI (continuous integration: the server that builds and tests every push) machine don't clash.

12.14 Testcontainers and `@ServiceConnection`

> **(needs Docker — not run.)** Docker is not running on the machine that built this book, so `testcontainers-demo` contains a `.tools_run_skip` marker. Its code was **compiled** (`mvn test-compile`) against Boot 4.1.1 and Testcontainers 2.0.5 but not executed.

What it is

Testcontainers is a Java library that starts real services (PostgreSQL, Kafka, Redis, ...) in throw-away **Docker containers** (Docker runs a program with everything it needs in an isolated, lightweight box called a container; the template it starts from is an **image**) from a test and removes them afterwards. `@ServiceConnection` (Boot 3.1) tells Boot to read the connection details (host, mapped port, user, password) straight from the container. You don't copy them into properties yourself.

Why it matters

H2 in PostgreSQL mode is still H2. `ILIKE`, `jsonb`, `ON CONFLICT`, partial indexes, `SKIP LOCKED`, exact numeric and collation (the rules for sorting and comparing text) behaviour, and Flyway migrations written for PostgreSQL can only be proven against PostgreSQL. A container gives every developer and CI run the same real database version, with no shared test server.

How it works

1. `@Testcontainers` (JUnit extension) finds `@Container` fields and starts them. **`Static`** fields start once per class, instance fields once per test.
2. The container picks a random free host port (a port on your real machine) mapped to 5432, PostgreSQL's port inside the container.
3. `@ServiceConnection` registers a `JdbcConnectionDetails` bean (for PostgreSQL) built from the running container. Boot's `DataSource` auto-configuration uses those details *instead of* `spring.datasource.*`.
4. After the tests the container is stopped. Testcontainers' "Ryuk" helper container also removes leftovers if the JVM crashes.

Code (not run)

`pom.xml` — note the **Testcontainers 2.x artifact names** (`testcontainers-<module>`; 1.x used plain `postgresql`, `junit-jupiter`):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-postgresql</artifactId>
  <scope>test</scope>
</dependency>
```

A repository method that uses PostgreSQL-only SQL:

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    // PostgreSQL-only SQL (ILIKE): H2 would not prove this works in production
    @Query(value = "select * from customer where email ilike concat('%', :part, '%')", nativeQuery = true)
    List<Customer> search(String part);
}
```

The test:

```
/** (needs Docker - not run in this book) A real PostgreSQL in a throw-away container. */
@DataJpaTest     // default Replace.NON_TEST: a @ServiceConnection database is a "test" one, so it is kept
@Testcontainers
class CustomerRepositoryPostgresTest {

    @Container                // started before the tests, stopped after (static = once per class)
    @ServiceConnection        // Boot reads host/port/user/password from the container
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");

    @Autowired CustomerRepository repo;

    @Test
    void ilikeWorksOnRealPostgres() {
        repo.save(new Customer("Ada@Example.com"));
        assertThat(repo.search("example")).extracting(Customer::getEmail).containsExactly("Ada@Example.com");
    }
}
```

In Testcontainers 2 the class is `org.testcontainers.postgresql.PostgreSQLContainer` and is no longer generic. The 1.x class `org.testcontainers.containers.PostgreSQLContainer<?>` still ships but is deprecated.

Containers as beans, reused by tests and by local development

```java
@TestConfiguration(proxyBeanMethods = false)
public class ContainersConfig {
    @Bean
    @ServiceConnection
    PostgreSQLContainer postgres() {
        return new PostgreSQLContainer("postgres:17-alpine");
    }
}
```

- Tests: `@SpringBootTest @Import(ContainersConfig.class)`. The container is a bean, so it is started with the context and **shared through the context cache** by every test class with the same configuration.
- Local development: a `main` in `src/test/java` starts the real app plus the containers:

```java
public class TestTcApplication {
    public static void main(String[] args) {
        SpringApplication.from(TcApplication::main).with(ContainersConfig.class).run(args);
    }
}
```

Run it from the IDE or with `mvn spring-boot:test-run` (Boot 3.1+). New developers get a working database without installing one.

`@ServiceConnection` **vs** `@DynamicPropertySource`

	`@ServiceConnection`	`@DynamicPropertySource`
Code	one annotation	`registry.add("spring.datasource.url", postgres::getJdbcUrl)` + user + password
Supported services	those Boot knows, e.g. JDBC and R2DBC databases, MongoDB, Redis, Kafka, RabbitMQ, Elasticsearch, Cassandra, Neo4j (full list in the Boot reference, "Testcontainers")	anything
Use when	the container is a supported type	custom containers (WireMock, your own image), custom properties

Trade-offs

- **Pro:** production-like behaviour, the same version everywhere, no shared test database.
- **Con:** needs Docker on every machine and CI agent. The first run pulls images, and each container start adds seconds. Mitigate with static/shared containers, the context cache, and CI image caching.

Common mistakes

- **One container per test class** without sharing → dozens of PostgreSQL starts. Share through a base class or `@Import(ContainersConfig.class)`.
- `image:latest` → a new major version breaks the build one morning. Pin the version you run in production.
- **Mixing** `@ServiceConnection` **with** `spring.datasource.url` **in** `application-test.properties` → the connection details win, and the property silently does nothing. Remove it to avoid confusion.

In the interview

Q: How do you test against a real database in Spring Boot? "Testcontainers with `@ServiceConnection`. A static `PostgreSQLContainer` field or a `@TestConfiguration` bean, and Boot takes the JDBC details from the container, so no URL properties are needed. I keep it in one shared configuration so the context cache reuses the same container, and I pin the image to the production version. For services Boot doesn't support, I use `@DynamicPropertySource`."

12.15 Profiles and properties in tests

The tools

Mechanism	Scope	Example
`src/test/resources/application-test.properties` + `@ActiveProfiles("test")`	classes that activate the profile	`FullStackTest` in this chapter
`@SpringBootTest(properties = "k=v")`	one test class	`MockMvcFullContextTest`
`@TestPropertySource(properties = "k=v")` or `(locations = "classpath:x.properties")`	one test class (inherited by subclasses)	`ExchangeRateClientTest`, `C4_OtherPropertyTest`
`@DynamicPropertySource`	one test class, runtime values	`FullStackTest`

Who wins? Measured

`PropertyPrecedenceTest` sets the same keys in several places. It uses the tiny `@JsonTest` slice, because slice annotations also have a `properties` attribute:

```
@JsonTest(properties = {"demo.a=from-properties-attribute", "demo.b=from-properties-attribute", "demo.d=from-properties-attribute"})
@TestPropertySource(properties = {"demo.a=from-@TestPropertySource", "demo.c=from-@TestPropertySource", "demo.d=from-
@TestPropertySource"})
@ActiveProfiles("test")
class PropertyPrecedenceTest {

    @DynamicPropertySource
    static void dynamic(DynamicPropertyRegistry r) {
        r.add("demo.a", () -> "from-@DynamicPropertySource");
        r.add("demo.b", () -> "from-@DynamicPropertySource");
        r.add("demo.c", () -> "from-@DynamicPropertySource");
    }

    @Value("${demo.a}") String a;
    ...
    @Value("${catalog.home-currency}") String fromProfileFile;   // from application-test.properties
}
```

Output:

```
all three set demo.a   -> from-@DynamicPropertySource
attribute vs dynamic   -> from-@DynamicPropertySource
@TestPropertySource vs dynamic -> from-@DynamicPropertySource
attribute vs @TestPropertySource -> from-@TestPropertySource
application-test.properties    -> catalog.home-currency=EUR
```

So, highest first: **`@DynamicPropertySource` → `@TestPropertySource` → the `properties` attribute of `@SpringBootTest`/slices → command-line arguments → system properties → environment variables → `application-{profile}.properties` → `application.properties`**. This matches the Spring Framework reference ("dynamic properties have higher precedence than `@TestPropertySource`"). The ordered list in the Boot reference shows `@TestPropertySource` above `@DynamicPropertySource`, but that is not what Boot 4.1.1 does. Chapter 3 has the full non-test list.

Common mistakes

- **A `src/test/resources/application.properties`** with the same name as the main one *shadows* the main file completely (the test classpath comes first), so every property in the main file disappears in tests. Use `application-test.properties` + `@ActiveProfiles("test")` to *add to* the main file.
- **Activating a different profile in every class** → a new context each time.
- **Production secrets in test profiles.** Tests should use fakes, containers or local stubs.

Self-check

1. Why prefer `application-test.properties` over a test `application.properties`? → The profile file is layered on top of the main file; a same-named file replaces it.
2. Which is applied last and wins: `@TestPropertySource` or `application-test.properties`? → `@TestPropertySource`.

12.16 Common mistakes — checklist

Symptom	Likely cause	Fix
"Unable to find a @SpringBootConfiguration"	test package is not under the main class's package	move the test, or `@SpringBootTest(classes = App.class)`
`No qualifying bean of type ...Service` in `@WebMvcTest`	services aren't in the slice	`@MockitoBean` or `@Import`
`No qualifying bean of type ...MockMvc/RestTestClient/TestRestTemplate` in `@SpringBootTest`	Boot 4 needs explicit opt-in	`@AutoConfigureMockMvc` / `@AutoConfigureRestTestClient` / `@AutoConfigureTestRestTemplate`
`@MockBean` doesn't compile after upgrading to Boot 4	removed	`@MockitoBean` / `@MockitoSpyBean`
Slice loads repositories/services	custom `@ComponentScan` on the main class	move scanning config to a separate `@Configuration`
Test passes, production fails with `LazyInitializationException`	test-managed transaction kept the session open	non-transactional integration test (ch08)
Data from one test visible in the next	real-port / non-transactional test didn't clean up	`@BeforeEach` cleanup, `@Sql` scripts
Suite gets slower every sprint (a fixed work period, often two weeks)	many unique context configurations, `@DirtiesContext`	base class, slices, cache DEBUG log
Random failures only on CI	fixed ports, shared databases, time or order dependence	`RANDOM_PORT`, containers or embedded DBs, fixed `Clock` bean, no static state

---

12.17 In the interview

Q: How do you test a Spring Boot REST API? "Three levels. Unit tests for the service logic with Mockito, no Spring. A `@WebMvcTest` per controller with `MockMvcTester` and `@MockitoBean` services to check mappings, validation, status codes and Problem Details. A few `@SpringBootTest`s: mock environment with `@AutoConfigureMockMvc` for wiring, and one or two `RANDOM_PORT` tests with `RestTestClient` for the real HTTP path. Repositories get `@DataJpaTest`, against PostgreSQL via Testcontainers when the SQL is database-specific."

- *`@SpringBootTest` vs `@WebMvcTest`?* → Whole context vs web slice only; the slice is faster and needs mocks for non-web beans.
- *What is `MockMvcTester`?* → Framework 6.2's AssertJ-based MockMvc API: `mvc.get().uri(..).exchange()`, `assertThat(result).hasStatusOk().bodyJson()...`.
- *What's new for testing in Spring Boot 4 / Framework 7?* → Per-technology test starters and moved packages; `@MockBean` removed in favour of `@MockitoBean`; explicit `@AutoConfigure...` for test clients; `RestTestClient`; context pausing; JUnit 6.

Q: How does Spring cache test contexts and what breaks the cache? → See §12.11: the key is the merged configuration; mocks, properties, profiles, imports, dynamic properties and `@DirtiesContext` cause new contexts; default max 32 (LRU).

Q: How do you test code that calls another service? → Unit: mock the client. Client class: `@RestClientTest` + `MockRestServiceServer`, or a local stub server via `@DynamicPropertySource`. End-to-end: stub server or WireMock container; consumer-driven contract tests (Pact; Spring Cloud Contract has been archived) to keep stubs honest (§16.10).

Q: Why might a `@Transactional` test give false confidence? → It rolls back everything, so commit-time behaviour (deferred constraints, `AFTER_COMMIT` listeners, lazy loading outside the session) never runs; and in `RANDOM_PORT` tests it doesn't even cover the server's transaction.

12.18 Types of API testing — and the Spring tool for each

What it is

§12.2 sorted tests by **how much of the container they start**. Teams, QA (quality assurance, i.e. testing) engineers and interviewers also sort them by **what question they answer**: "does it start?", "does it follow the spec?", "does it survive 5,000 users?", "can an attacker break it?". These names — smoke, functional, integration, contract,

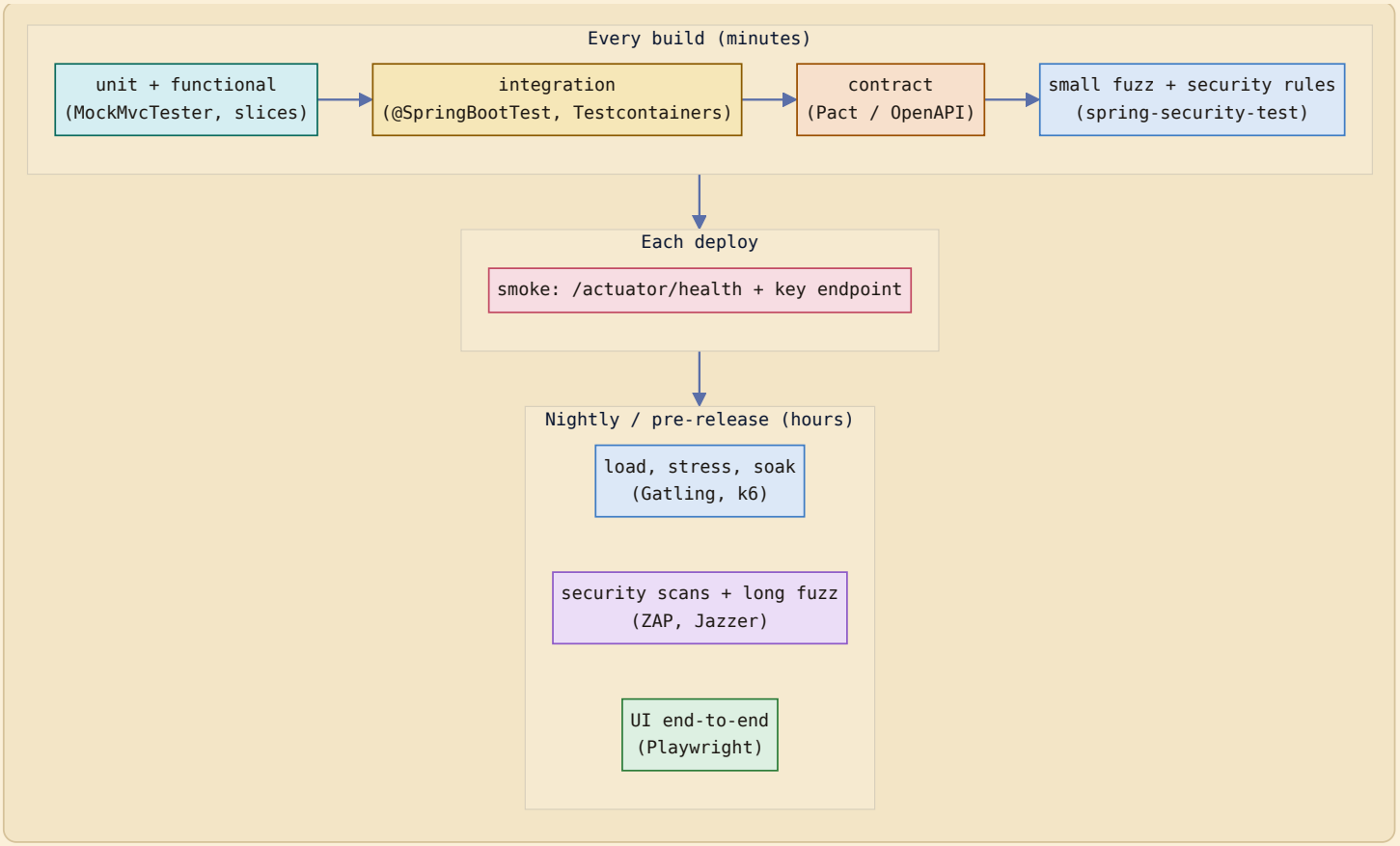
regression, load, stress, security, fuzz, UI — describe the *goal* of a test, not its size. Many are built from the same Spring tools you have seen in this chapter; a few need tools outside Spring.

The problem it solves

A suite of 400 green `@WebMvcTest`s can still ship an API that falls over at 300 requests per second, breaks the mobile app because a field was renamed, or leaks another user's order to anyone who changes an ID in the URL. Naming the test types is a checklist for *which risks you have not tested yet*.

How it works — the matrix

Type	Question it answers	When it runs	Spring / Java tool	In this book
Smoke	Does the app start and answer at all?	every build; right after every deploy (release to a server)	one `@SpringBootTest` that loads the context; after deploy, a script hitting `/actuator/health` (Spring Boot Actuator's health-check URL) and one key endpoint	§12.6, ch15 health
Functional	Does each endpoint behave as the spec says (status, body, errors, validation)?	every build	`@WebMvcTest` + `MockMvcTester`, `@MockitoBean` for services	§12.7, §12.8
Integration	Do the real parts work together: DB, SQL, transactions, HTTP clients, messaging?	every build	`@SpringBootTest`, `@DataJpaTest`, `@RestClientTest`, Testcontainers + `@ServiceConnection`	§12.6, §12.7, §12.14
Contract	Do provider and consumers still agree on the API's shape?	provider and consumer builds	consumer-driven contracts (Pact), OpenAPI (a standard machine-readable description of an HTTP API) schema validation in tests	§16.10
Regression	Did this change break something that used to work?	every build	not a separate tool: the whole automated suite, plus one new test for every bug fixed	whole chapter
Load	Does it meet its latency/throughput (time per request / requests per second) goal (SLO, service level objective) at expected peak?	before releases, nightly	Gatling (Java/Scala DSL), k6, JMeter (load-generating tools) against a production-like environment; watch Micrometer (Spring's metrics library) metrics	ch15
Stress / spike / soak	Where does it break and how; does it recover from a sudden burst; does it leak over 8 hours?	before big launches	same tools, different load shape	—
Security	Are unauthenticated users rejected, authorisation rules enforced, inputs sanitised?	every build (+ periodic scans)	`spring-security-test` (`@WithMockUser`, `jwt()`), OWASP ZAP (an automated web security scanner) scans, dependency scanning for known-vulnerable libraries (OWASP Dependency-Check, Dependabot)	10.12, 11.6
Fuzz	Does random, malformed or hostile input ever crash it (5xx = server-error status codes, hang, leak)?	every build (small), nightly (large)	a seeded random loop over `MockMvcTester` (below), property-based testing (jqwik; generates many inputs and checks a rule holds for all), coverage-guided fuzzing (Jazzer; mutates inputs to reach new code paths), OpenAPI-driven fuzzers (Schemathesis)	below
UI / end-to-end	Does a real user journey work through the browser?	before release	Playwright or Selenium (tools that drive a real browser) against a `RANDOM_PORT` app or staging (a production-like pre-release environment)	§12.13



Two terms that are often mixed up: **load** testing checks behaviour *at* the expected peak (e.g. 2,000 requests/s with **p99** under 200 ms — p99 is the latency that 99% of requests beat; see System Design Fundamentals), **stress** testing pushes *beyond* it to find the breaking point and check the failure is graceful (429s — "Too Many Requests" responses — and **back-pressure**, i.e. pushing back on callers when overloaded, not an out-of-memory crash), and **soak** testing holds normal load for hours to reveal leaks and slow degradation. A load test on a laptop measures the laptop — run it against an environment sized like production.

Seeing it: a fuzz test — `ProductApiFuzzTest`

The test sends 500 requests to the web slice: `POST /api/products` bodies built from valid and hostile values (empty strings, `null`, wrong types, `1e400`, SQL-injection (input crafted to change the meaning of a database query) and `<script>` strings, a 5,000-character SKU, truncated JSON, 200 levels of nesting) and `GET` requests with random path segments. It asserts one property: **the API never answers 5xx**. The `Random` **seed** (the starting number that determines the whole random sequence) is fixed, so any failure can be replayed exactly. It reuses the same configuration as `ProductControllerWebMvcTest`, so it costs no extra context start.

```
Random rnd = new Random(12);                                  // simplified excerpt: POST branch only
for (int i = 0; i < 500; i++) {
    String body = rnd.nextInt(6) == 0 ? pick(rnd, JUNK)
            : "{\"sku\":%s,\"name\":%s,\"price\":%s}".formatted(pick(rnd, SKUS), pick(rnd, NAMES), pick(rnd, PRICES));
    int status = mvc.post().uri("/api/products").contentType(MediaType.APPLICATION_JSON).content(body)
            .exchange().getResponse().getStatus();
    assertThat(status).as("request %d", i).isLessThan(500);   // 4xx is fine; 5xx is a bug
}
```

```
fuzz: 500 requests, status histogram {201=5, 400=395, 404=100}
```

Bean Validation (`@Valid` plus constraint annotations such as `@NotBlank`), Jackson's parse errors and the Problem Details handler turned every bad input into a 400 or 404. If an input had produced a 500, the fix would be a validation rule or an `@ExceptionHandler`, plus a normal functional test for that exact input (a new **regression** test).

When not to use it

- Don't build a separate "smoke suite" that duplicates functional tests; a context-load test plus a post-deploy health check is enough.
- Don't run load or stress tests on every commit; they need a stable, production-like environment and minutes to hours.
- Don't write end-to-end UI tests for rules a functional test can prove; keep UI tests to a handful of critical journeys.

Common mistakes

- **Only happy-path functional tests** → add negative cases: 400, 401, 403, 404, 409, and "user A reads user B's order" (broken object-level authorisation, OWASP API Security Top 10 #1; OWASP is a non-profit that publishes widely used security checklists).
- **Load testing without a goal** → state the target first (requests/s, p95/p99 latency, error rate), then test against it.
- **Unseeded random tests** → a failure you can't reproduce; always log or fix the seed.
- **Mocking the other service in every test and never checking the real contract** → green builds, broken integration; add contract tests.

In the interview

> **30-second answer:** "I map test types to risks. Functional tests with `@WebMvcTest` and `MockMvcTester` check each endpoint's contract, including error cases. Integration tests with `@SpringBootTest` and Testcontainers check the real database and wiring. Contract tests with Pact keep services compatible. Security tests use `spring-security-test` for 401/403 rules plus a ZAP scan. A smoke check runs after every deploy. Load, stress and soak tests with Gatling or k6 run before releases against a production-like environment, and I fuzz inputs to make sure nothing returns a 5xx."

- *Regression test?* → Any automated test that guards behaviour that once worked; the rule is one new test per fixed bug.
- *What does a fuzz test assert if you don't know the expected output?* → Properties that must always hold: no 5xx, no hang, no stack trace in the body, response matches the schema.

Self-check

1. Load vs stress test? → Load checks the SLO at expected peak; stress goes past it to find the breaking point and failure mode.
2. Which test type catches a renamed JSON field before the mobile app breaks? → A contract test.
3. Why fix the random seed in a fuzz test? → So a failing input can be replayed and turned into a regression test.

TL;DR

Smoke = does it start; functional = does each endpoint follow the spec (`@WebMvcTest`); integration = real parts together (`@SpringBootTest`, Testcontainers); contract = services agree (Pact); regression = the suite guarding old behaviour; load/stress/soak = performance at, beyond and over time (Gatling, k6); security and fuzz = hostile users and inputs (`spring-security-test`, ZAP, seeded random inputs); UI = a few critical browser journeys (Playwright).

Sources: Spring Framework reference (MockMvc, TestContext), Spring Boot reference (testing, Actuator health), OWASP API Security Top 10 (2023), OWASP ZAP docs, Gatling and k6 documentation, Pact documentation, jqwik and Jazzer user guides.

12.19 Chapter TL;DR

- **Pyramid**: many Spring-free unit tests (ms) → slices for layer behaviour (~0.1 s context) → a few `@SpringBootTest`s (~0.7 s context) → very few real-port tests.
- **Boot 4**: test starters per technology (`spring-boot-starter-webmvc-test`, `-data-jpa-test`, `-restclient-test`, ...); slice annotations moved packages; JUnit 6, AssertJ, Mockito inside `spring-boot-starter-test`.
- **TestContext Framework**: `SpringExtension` + listeners + a JVM-wide **context cache** keyed by the merged configuration.
- **`@SpringBootTest`**: `MOCK` (default), `RANDOM_PORT`, `DEFINED_PORT`, `NONE`; add `@AutoConfigureMockMvc` / `@AutoConfigureRestTestClient` explicitly in Boot 4; `@TestConfiguration` adds test beans.
- **Slices**: `@WebMvcTest` (127 vs 253 beans here), `@DataJpaTest` (embedded DB, rolled back, SQL shown), `@JsonTest`, `@RestClientTest` + `MockRestServiceServer`.
- **`MockMvcTester`**: AssertJ assertions on MockMvc results; **`RestTestClient`** (Framework 7.0): one fluent client for MockMvc or a real server.
- **`@MockitoBean` / `@MockitoSpyBean`** replace the removed `@MockBean` / `@SpyBean`; reset after each test; part of the cache key.
- **Caching demo**: 7 classes, 4 context builds; hits cost 0 ms, misses ~0.5 s. Mocks, properties and `@DirtiesContext` cause misses. Max 32 contexts; Framework 7 **pauses** inactive ones.
- **`@DynamicPropertySource`**: lazy suppliers for runtime values (random ports), highest precedence.
- **Testcontainers + `@ServiceConnection`**: real PostgreSQL with zero URL config (needs Docker — not run here); TC 2 artifacts `testcontainers-postgresql`; containers as beans also power `mvn spring-boot:test-run`.

- **Test types by goal** (§12.18): smoke, functional, integration, contract, regression, load/stress/soak, security, fuzz, UI — map each risk to a type; fuzz with a fixed seed and assert "never 5xx".
- Next: Chapter 13 — Reactive programming and WebFlux.